COM 3401
DATA COMMUNICATION AND NETWORKING

Lecture 07

ERROR DETECTION AND CORRECTION

P. Keerthishan
Temporary Assistant Lecturer
Dept. of Computing
Faculty of Applied Sciences, RUSL

Topics covered

- Types of Errors
- Forward Error Control
- Backward Error Control
- Error Detection
- Error Correction

Reference:

William Stallings, *Data and Computer Communications*, 8th Edition

Andrew S. Tanenbaum, *Computer Networks,* 5th Edition

Kurose and Ross, Computer Networking – A Top-Down Approach, 6th Edition

7.1
Understanding Errors



Why Do Transmission Errors Occur?

- A transmitted bit can be changed during transmission.
- Possible causes:
  - Noise
  - Signal attenuation
  - Interference
  - Other transmission impairments
- Example:

Transmitted	**1**	**0**	**1**	**1**	**0**	**1**	**0**	**0**
Received	**1**	**0**	**1**	**0**	**0**	**1**	**0**	**0**

- One bit has changed.

What Is a Bit Error?

- **Bit error:** A transmitted bit is received with the opposite value.

Sent	Received
0	1
1	0

- **Important:**
  - The receiver initially does not know what the original bit was.

7.2
Types of Errors



Types of Errors

There are **two general types of errors**:

1. Single-Bit Error

- Only one bit is altered.

Transmitted	1	0	1	1	0	1	0	0
Received	1	0	1	0	0	1	0	0

2. Burst Error

- A group/sequence of bits is affected.

Transmitted	1	0	1	1	0	1	0	0	0	1	1	1	0	1	0	1
Received	1	0	1	0	1	0	1	1	1	0	1	0	1	0	1	1

Single-Bit Error

- An isolated bit is altered.
- Nearby bits are not affected.
- Example:

Transmitted	**1**	**0**	**1**	**1**	**0**	**1**	**0**	**0**
Received	**1**	**0**	**1**	**0**	**0**	**1**	**0**	**0**

- **Possible cause:**
  - Small/random deterioration in the signal-to-noise ratio.

❖ single-bit error can occur when white noise causes enough signal deterioration to confuse the receiver's decision about one bit.

Burst Error

- A group of bits within a span is affected.
- Important:
  - **Not every bit inside the burst must necessarily be wrong.**
- Example:

Transmitted	**1**	**0**	**1**	**1**	**0**	**1**	**0**	**0**	**0**	**1**	**1**	**1**	**0**	**1**	**0**	**1**
Received	**1**	**0**	**1**	**0**	**1**	**0**	**1**	**1**	**1**	**0**	**1**	**0**	**1**	**0**	**1**	**1**

- Errors occur in a cluster.
- More common than isolated errors.
- More difficult to handle.
- Can be caused by impulse noise or fading.
- Effect becomes greater as data rate increases.

Why Does Data Rate Affect Burst Errors?

- Suppose a noise event lasts **1 ms**.

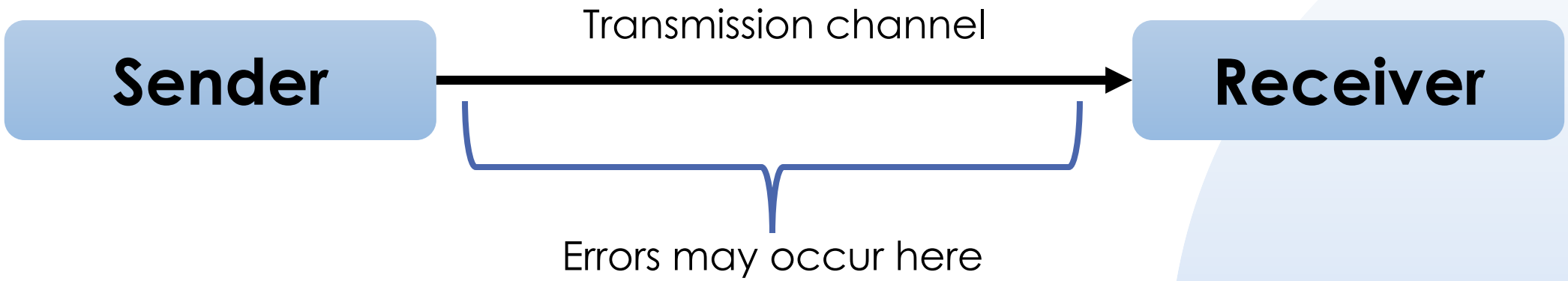
Data rate	Bits affected
10 Mbps	10 bits
100 Mbps	100 bits

- **Key idea**
  - The same physical disturbance can affect more bits when the data rate is higher.

7.3
Dealing with Errors



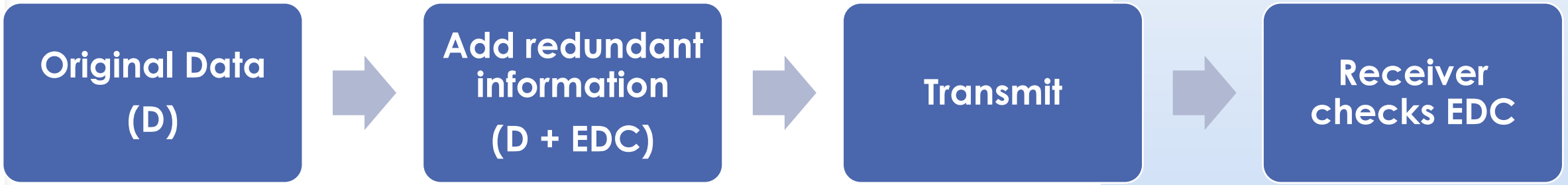
Error-Control Problem



- The receiver needs to answer:
  - **"Is the received data the same as what was transmitted?"**
- If not:
  - **"How can we recover the correct data?"**

Adding Redundancy

- basic setting as data **D** being augmented with error-detection/correction bits (**EDC**), transmitted, possibly corrupted, and then checked at the receiver.



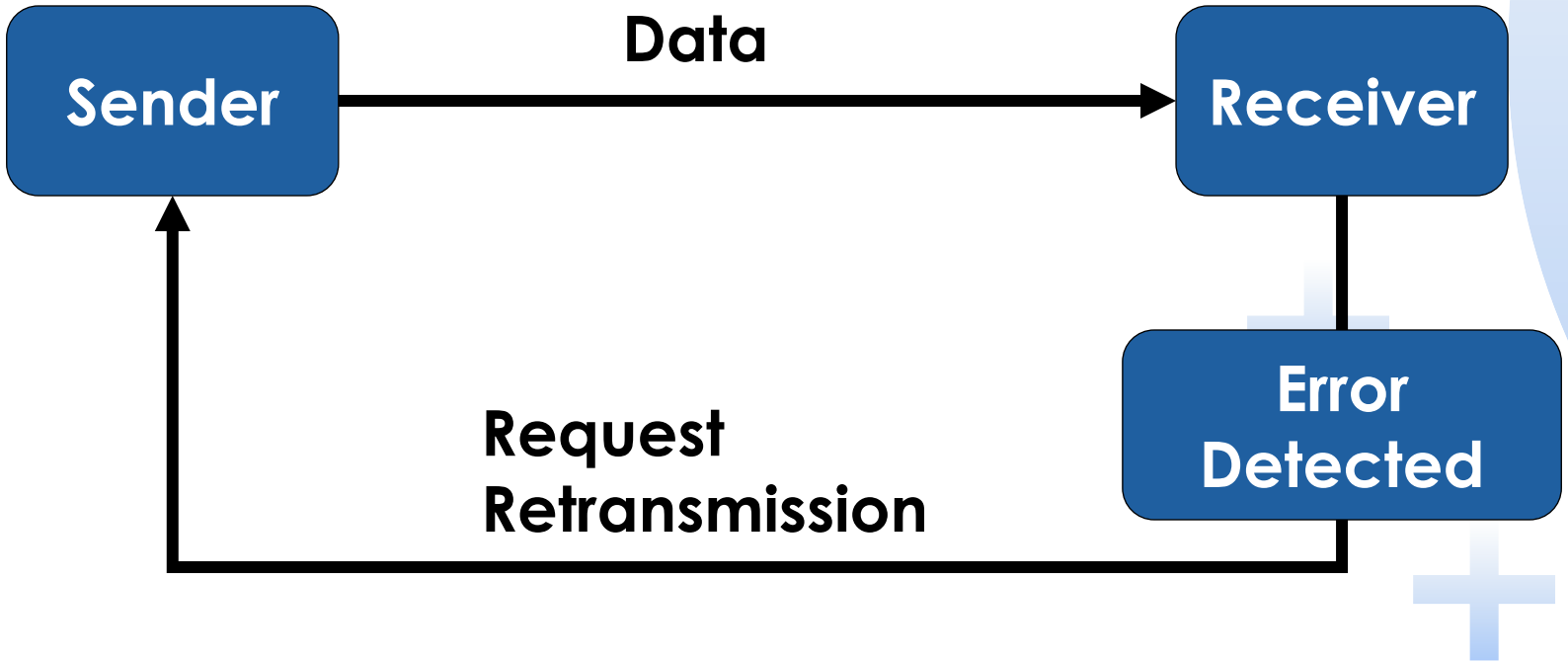
- The extra bits give the receiver additional information about the original data.
  - **Redundancy makes error detection/correction possible.**

Two Basic Error-Control Strategies

There are two fundamental approaches:

1. Error Detection + Retransmission

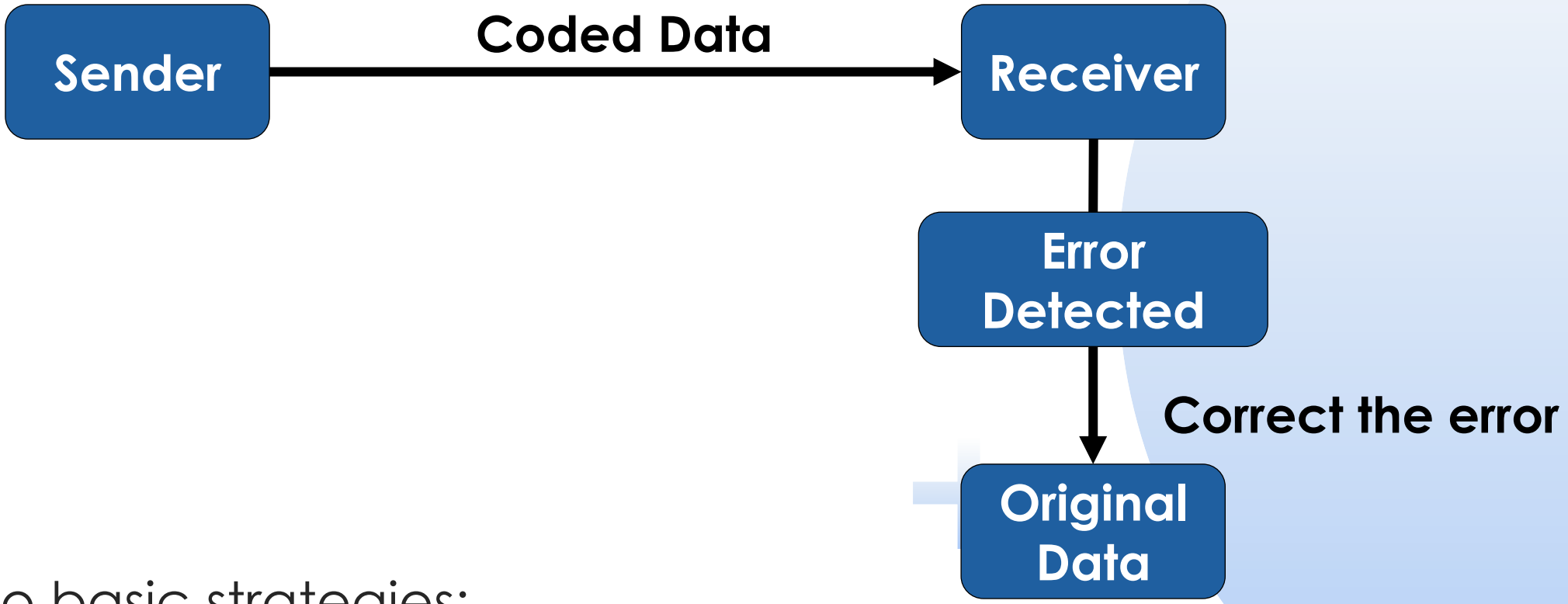
- Receiver detects an error but does not necessarily know how to fix it.



Two Basic Error-Control Strategies

2. Error Correction

Receiver uses redundant information to determine the original data.



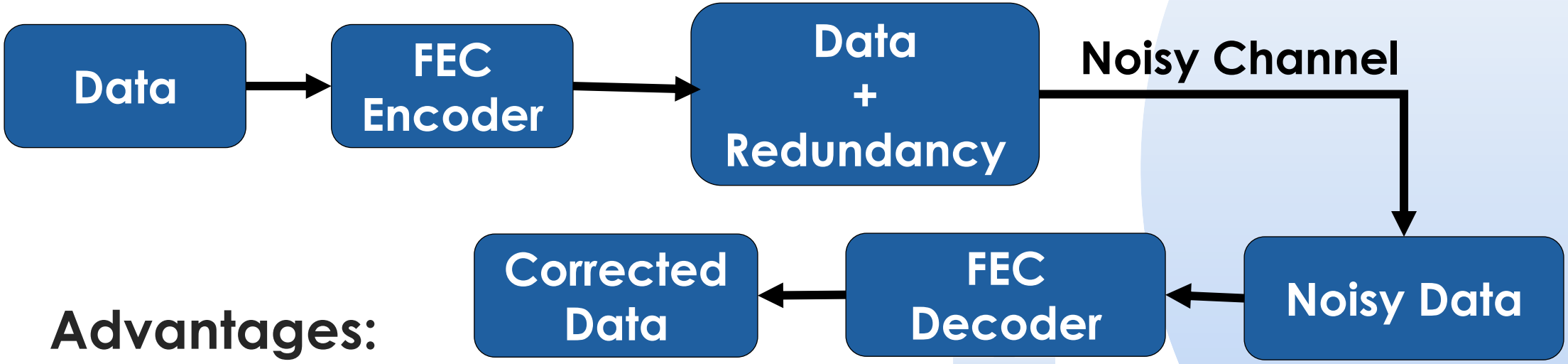
- Two basic strategies:
  - add enough redundancy to **deduce the transmitted data**, or add enough to **detect an error and request**

7.4
Forward Error Control



Forward Error Correction (FEC)

- The sender adds enough redundant information so that:
  - The receiver can detect AND correct certain errors **without retransmission.**



- **Advantages:**
  - No retransmission required
  - Immediate correction
  - Useful when retransmission is expensive or slow

Where Is FEC Useful?

- **FEC is particularly useful when:**
  - Channel has relatively high error rates
  - Retransmission is expensive
  - Round-trip delay is large
  - Real-time communication is important

- **Examples:**
  - Wireless links
  - Audio CDs / audio storage and playback
  - Deep-space communication

FEC: Advantages vs Cost

Advantages	Cost
Corrects errors at receiver	Extra redundant bits
No retransmission	More processing
No waiting for ACK/NAK	Reduced effective efficiency
Useful for long-delay links	More complex coding

- FEC trades additional redundancy for reduced retransmissions.

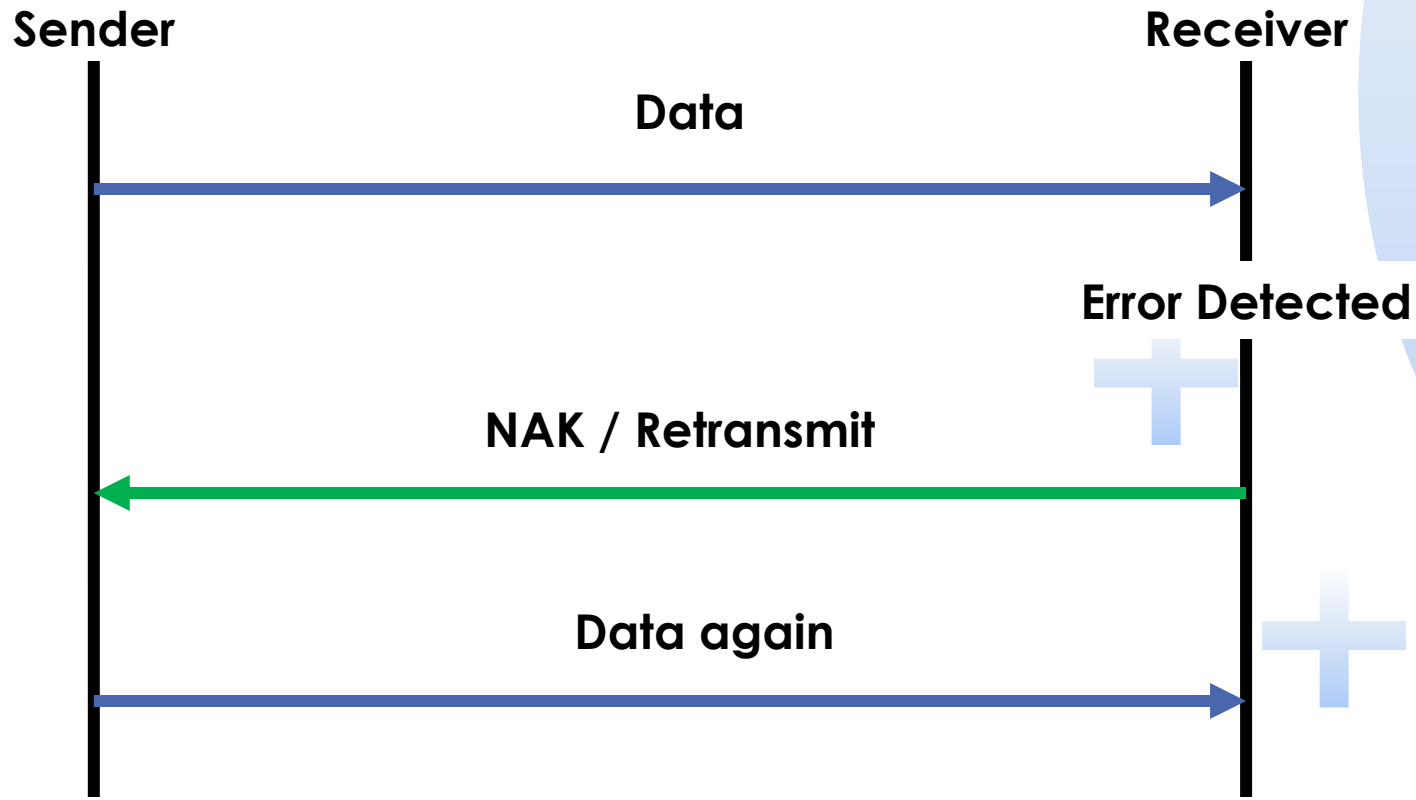
7.5
Backward Error Control



Backward Error Control

Instead of correcting the error at the receiver:

1. Receiver detects an error.
2. Receiver informs sender.
3. Sender retransmits the data.



FEC vs Backward Error Control

FEC	Backward Error Control
Receiver corrects error	Receiver detects error
No retransmission needed	Retransmission required
More redundancy	Less redundancy
More processing	Simpler detection
Good for noisy/long-delay links	Good when errors are relatively rare
Immediate correction	Requires feedback

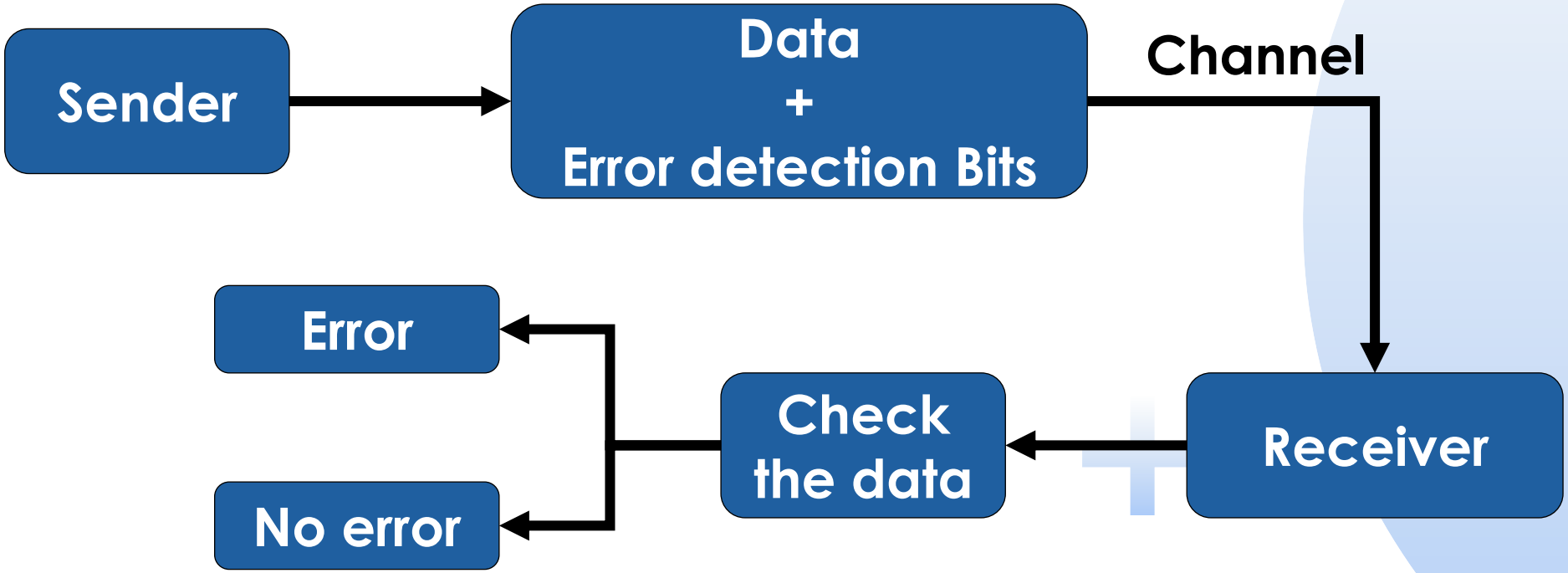
- **Key idea**
  - Reliable channel → detection + retransmission may be efficient.
  - Noisy/long-delay channel → FEC may be preferable.

7.6
Error Detection



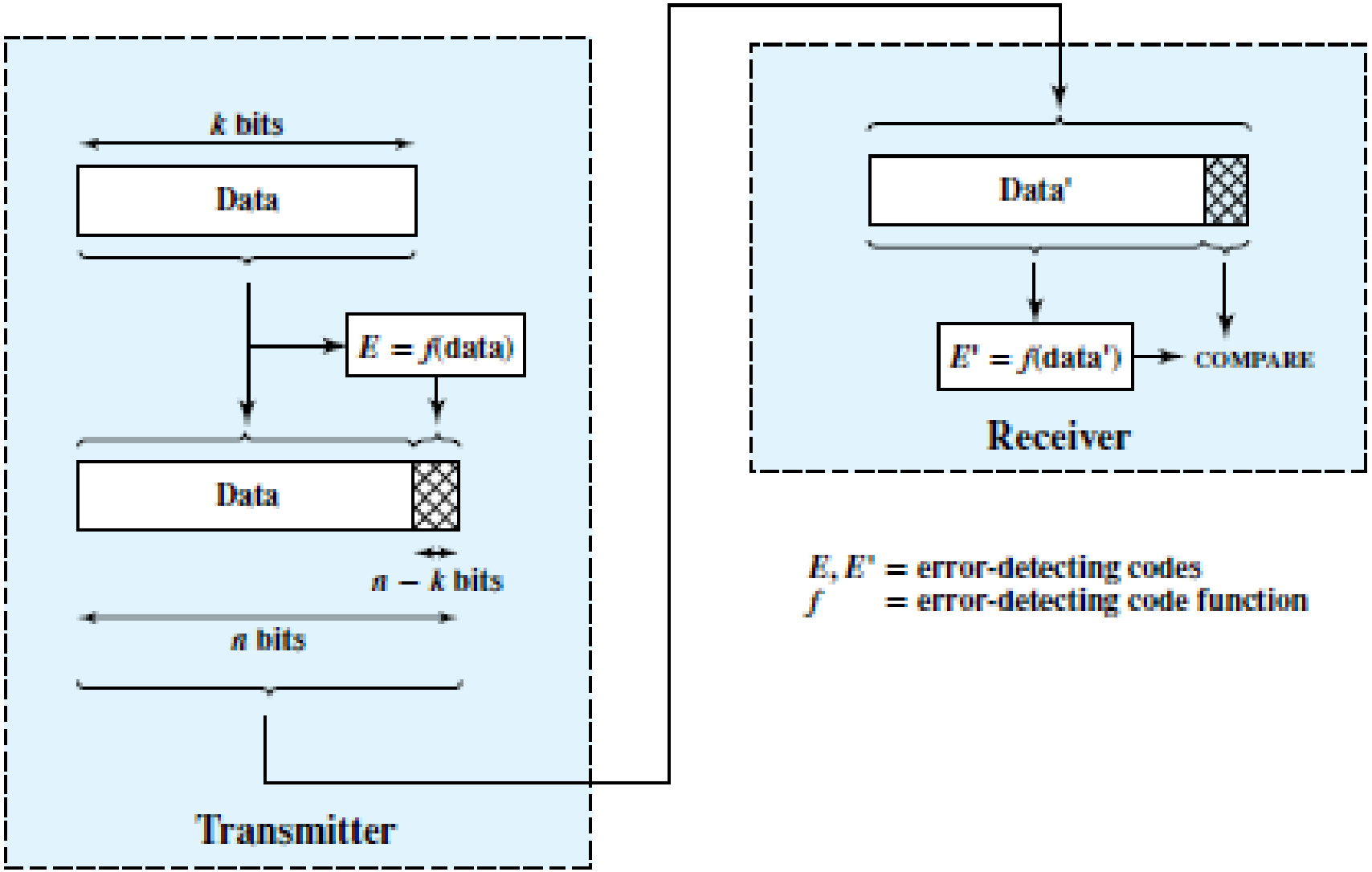
Error Detection

- **Error detection asks:**
  - "Has the received data been corrupted?"
- The sender adds **redundant bits**.



- **Important:**
  - Detection **does not necessarily identify which bit is wrong**.

Error Detection Process



Main Error-Detection Techniques

The three techniques covered are:

1. Parity Checks

- Simple and low overhead.

2. Checksums

- Treat data as groups of bits and calculate a sum.

3. Cyclic Redundancy Check (CRC)

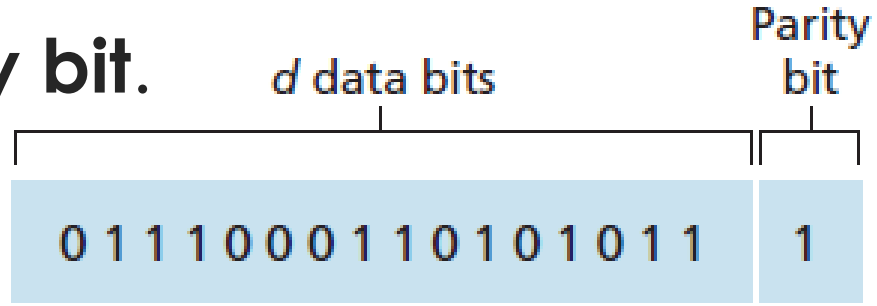
- Uses modulo-2 arithmetic and a generator.

7.7
Parity



Single-Bit Parity

The sender adds **one parity bit**.



1. Even parity

- Make the total number of 1s **even**.
- Example:

Data : 1 0 1 1 0 1 0

No. of ones = 4

Parity = 0

Transmitted : 1 0 1 1 0 1 0 0

Single-Bit Parity

2. Odd parity

- Make the total number of 1s **odd**.
- Example:

Data : 1 0 1 1 0 1 0

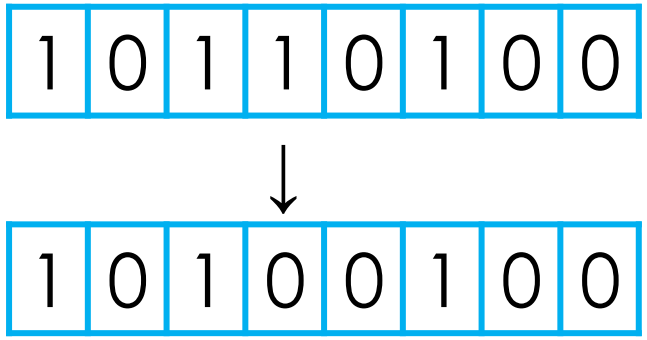
No. of ones = 4

Parity = 1

Transmitted : 1 0 1 1 0 1 0 1

How Does Parity Detect an Error?

- Suppose even parity is used,

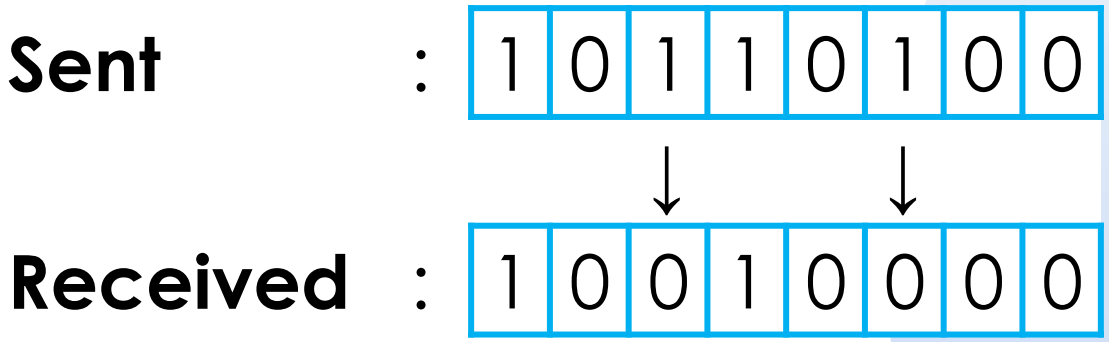


- Count the 1s.
- If the number of 1s is **odd**:
  - **Error detected**

How Does Parity Detect an Error?

But...

- If **two bits** change (consider even parity):



- The parity may still be correct.
  - **Two-bit errors can go undetected.**

Limitation of Single Parity

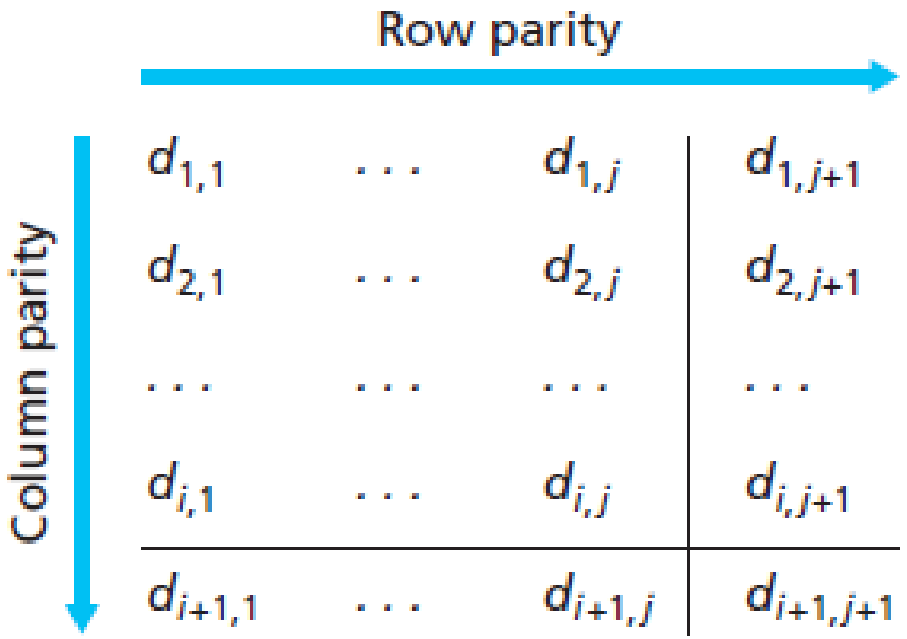
- **Single parity is simple, but limited.**
- **Detects reliably:**
  - Single-bit errors
  - More generally, odd numbers of flipped bits
- **May fail to detect:**
  - Even numbers of flipped bits
  - Some burst errors

7.8
Two-Dimensional Parity



Two-Dimensional Parity

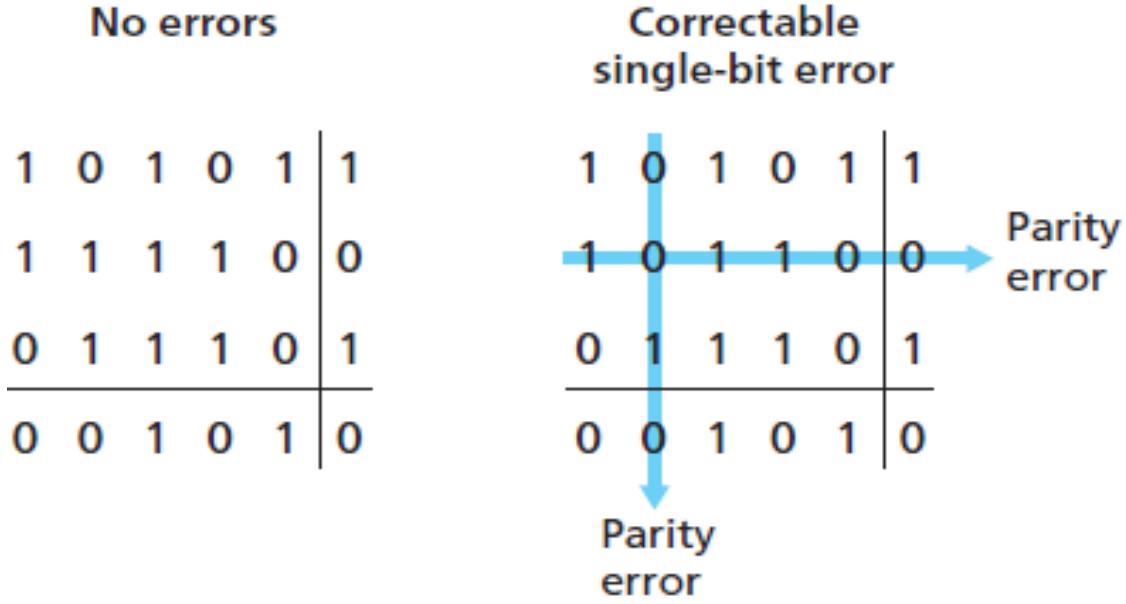
- Instead of one parity bit:
  - Calculate parity for **each row AND each column**.
- **At receiver**
  - Check every row
  - Check every column
  - Locate the intersection of the failed row and column



Two-Dimensional Parity Can Correct!

Two dimensional even parity

- Suppose:



- The intersection identifies the corrupted bit.

Therefore:

- **Single parity → detection**
- **Two-dimensional parity → can locate and correct a single-bit error**

7.9 Checksum



Checksum

Basic idea

1. Divide data into fixed-size words.
2. Add the words.
3. Generate a checksum from the result.
4. Transmit checksum with data.
5. Receiver performs the same check.

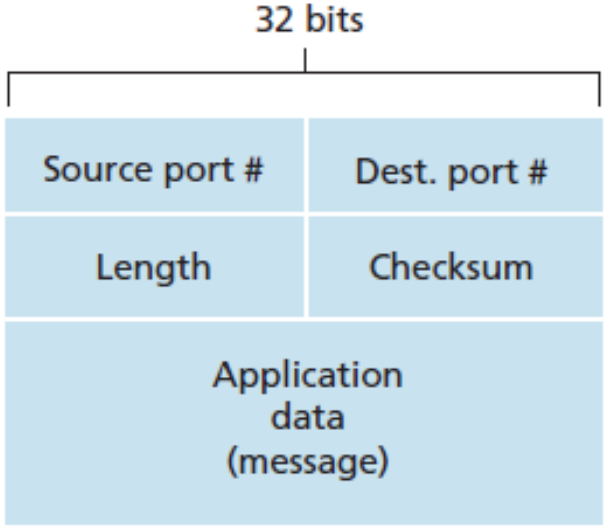
Internet Checksum

Process:

- Data is treated as **n bits** (commonly 16-bit words)
- Words are summed
- 1's complement is taken
- Result is placed in checksum field
- Receiver performs the corresponding check (if the final complemented sum (including checksum), equals zero)

Used in

- UDP
- TCP
- IP header checksum



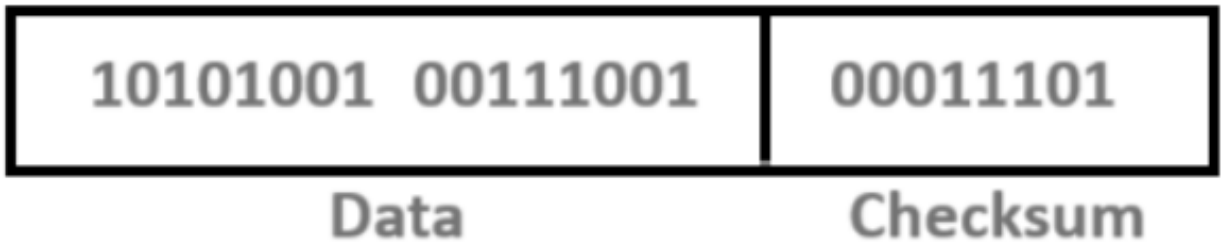
Checksum Example

If the data unit to be transmitted is 10101001 00111001 ,

At Sender site:

Subunit 1	1	0	1	0	1	0	0	1
Subunit 2	0	0	1	1	1	0	0	1
Sum	1	1	1	0	0	0	1	0
Checksum (Sum's complement)	**0**	**0**	**0**	**1**	**1**	**1**	**0**	**1**

Data transmitted to the receiver



Checksum Example

At receiver site:

Subunit 1	1	0	1	0	1	0	0	1
Subunit 2	0	0	1	1	1	0	0	1
Checksum	0	0	0	1	1	1	0	1
Sum	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**
Sum's complement	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**

Result is zero; it means no error.

7.10 CRC



Cyclic Redundancy Check (CRC)

- A powerful error-detection technique widely used in computer networks.
- Sender and receiver agree on a:
  - **Generator G**
- The sender calculates **R**, the CRC bits.
- The resulting bit pattern is designed to be **exactly divisible by G** using modulo-2 arithmetic.

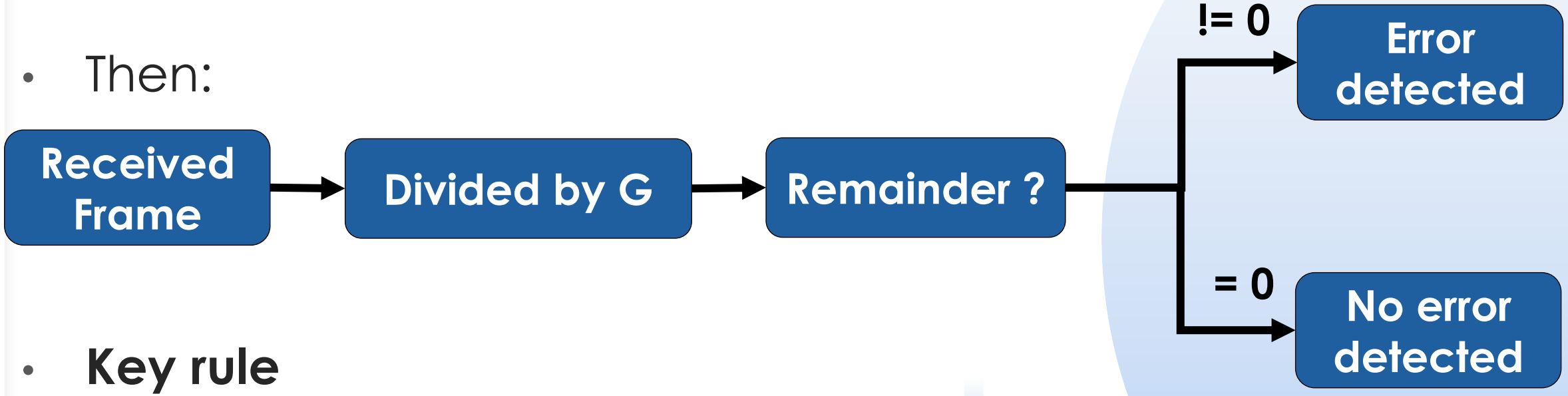
CRC: Sender Side

For a data block **D**:

1. Choose generator G
2. Append r zeros to D; where r = count of G - 1
3. Divide using modulo-2 division
4. Obtain remainder R
5. Replace the zeros with R
6. Transmit D + R

CRC: Receiver Side

- Receiver receives:
  - **D + R**
- Then:



- **Key rule**
  - Remainder = 0 → no error detected
  - Remainder ≠ 0 → error detected

CRC Uses XOR

- CRC uses **modulo-2 arithmetic**.
- In modulo-2 arithmetic:
  - 0 + 0 = 0
  - 0 + 1 = 1
  - 1 + 0 = 1
  - 1 + 1 = 0
- There are:
  - No carries
  - No borrows
- Therefore:
  - **Modulo-2 addition/subtraction = XOR**

$$\begin{array}{r} 1111 \\ +1010 \\ \hline 0101 \end{array} \qquad \begin{array}{r} 1111 \\ -0101 \\ \hline 1010 \end{array}$$

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

CRC - Example

Suppose:

Data	=	1	1	0	1
Generator	=	1	0	1	1

Step 1: Append zeros

- Generator has 4 bits, so we append 3 zeros to the data

Data	=	1	1	0	1			
Append 3 zeros	=	1	1	0	1	0	0	0

- So, we divide:

1	1	0	1	0	0	0	÷	1	0	1	1

CRC: Sender Side

Step 2: Binary division using XOR

- CRC uses XOR instead of normal subtraction

							1	1	1	1
1	0	1	1	1	1	0	1	0	0	0
				1	0	1	1			
				0	1	1	0	0		
					1	0	1	1		
					0	1	1	1	0	
						1	0	1	1	
						0	1	0	1	0
							1	0	1	1
							0	0	0	1

- Final remainder = 001

CRC: Sender Side

Step 3: From the transmitted codeword

Original data	=	1	1	0	1			
CRC remainder	=					0	0	1
Transmitted data	=	1	1	0	1	0	0	1

Step 4: What does the receiver do?

- The receiver divides the received codeword by the **same generator**:

1	1	0	1	0	0	1

÷

1	0	1	1

- If the remainder is

0	0	0

- No error detected

Why Is CRC Stronger Than Parity?

Parity	CRC
Very simple	More complex
Very little overhead	More check information
Limited error detection	Strong error detection
Can miss even-bit errors	Detects a much broader range of errors
Simple hardware/software	Efficient hardware implementation

Key idea:

- CRC provides much stronger error detection than a simple parity bit.

7.11
Error Correction



Error Detection vs Error Correction

Error Detection

- Receiver determines:
  - **"Something is wrong."**
- But may not know exactly what.

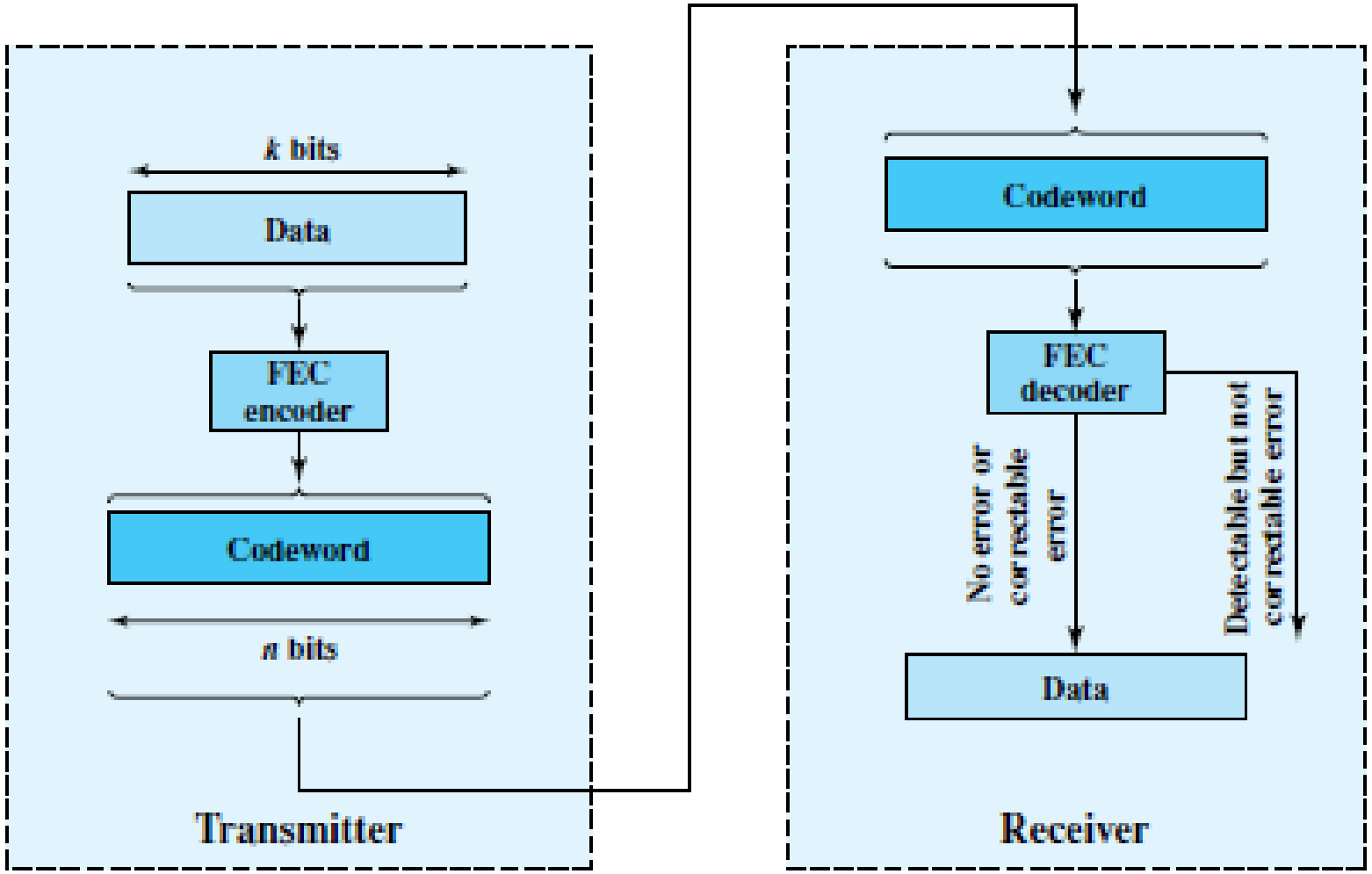
Data ➡️ Check ➡️ ERROR

Error Correction

- Receiver determines:
  - **"Something is wrong AND this is the most likely original data.“**

Data ➡️ Code ➡️ Error ➡️ Decode ➡️ Corrected Data

Error Correction Process



Redundancy Makes Correction Possible

- Suppose we have:
  - Data → Encoding → Codeword
- Instead of transmitting:
  - 00
  - 01
  - 10
  - 11
- we may assign longer codewords:
  - 00 → 00000
  - 01 → 00111
  - 10 → 11001
  - 11 → 11110
- Now the valid codewords are separated from each other.
- If an invalid pattern is received, the receiver can determine which valid codeword is **closest**.

7.12
Hamming Distance



Hamming Distance

- **Hamming distance** between two binary sequences:
  - Number of bit positions in which they differ.

V1	**=**	**0**	**1**	**1**	**0**	**1**	**1**
V2	**=**	**1**	**1**	**0**	**0**	**0**	**1**

↑ ↑ ↑

- They differ in **3 positions**.
- Therefore:
  - **d(v1,v2) = 3**

How Hamming Distance Help Correction

- Need to transmit data of length k-bits
- We use n-bit codeword
- For k=2 and n=5,

Data Block	Codeword
00	00000
01	00111
10	11001
11	11110

- Suppose we receive **00100**, not a valid code word.

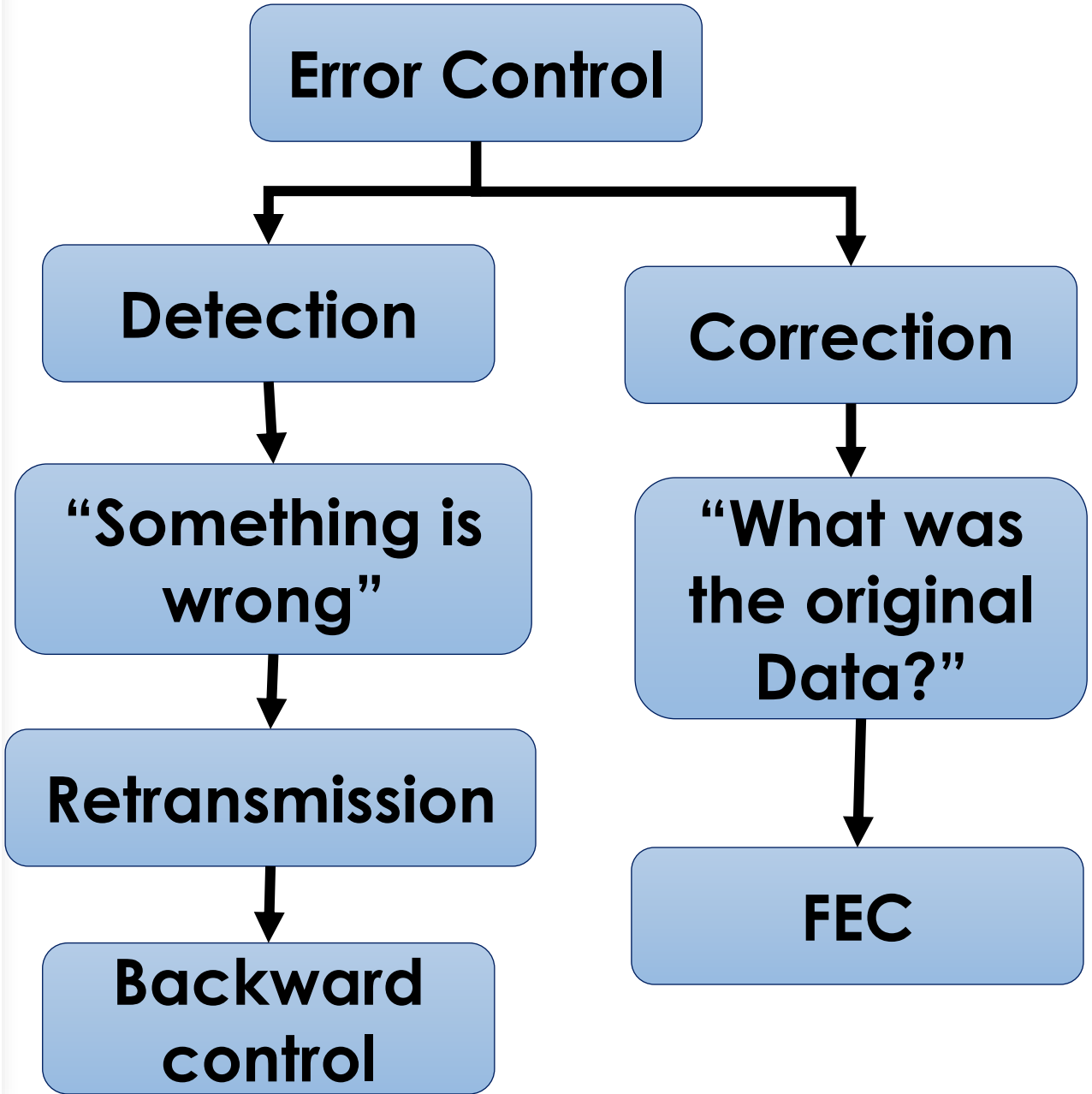
$$d(00000, 00100) = 1; \quad d(00111, 00100) = 2;$$
$$d(11001, 00100) = 4; \quad d(11110, 00100) = 3$$

- The closest valid codeword is **000000**. We deduce that the data most sent was 00.

7.13
Error Control Comparison



Complete Error-Control Picture



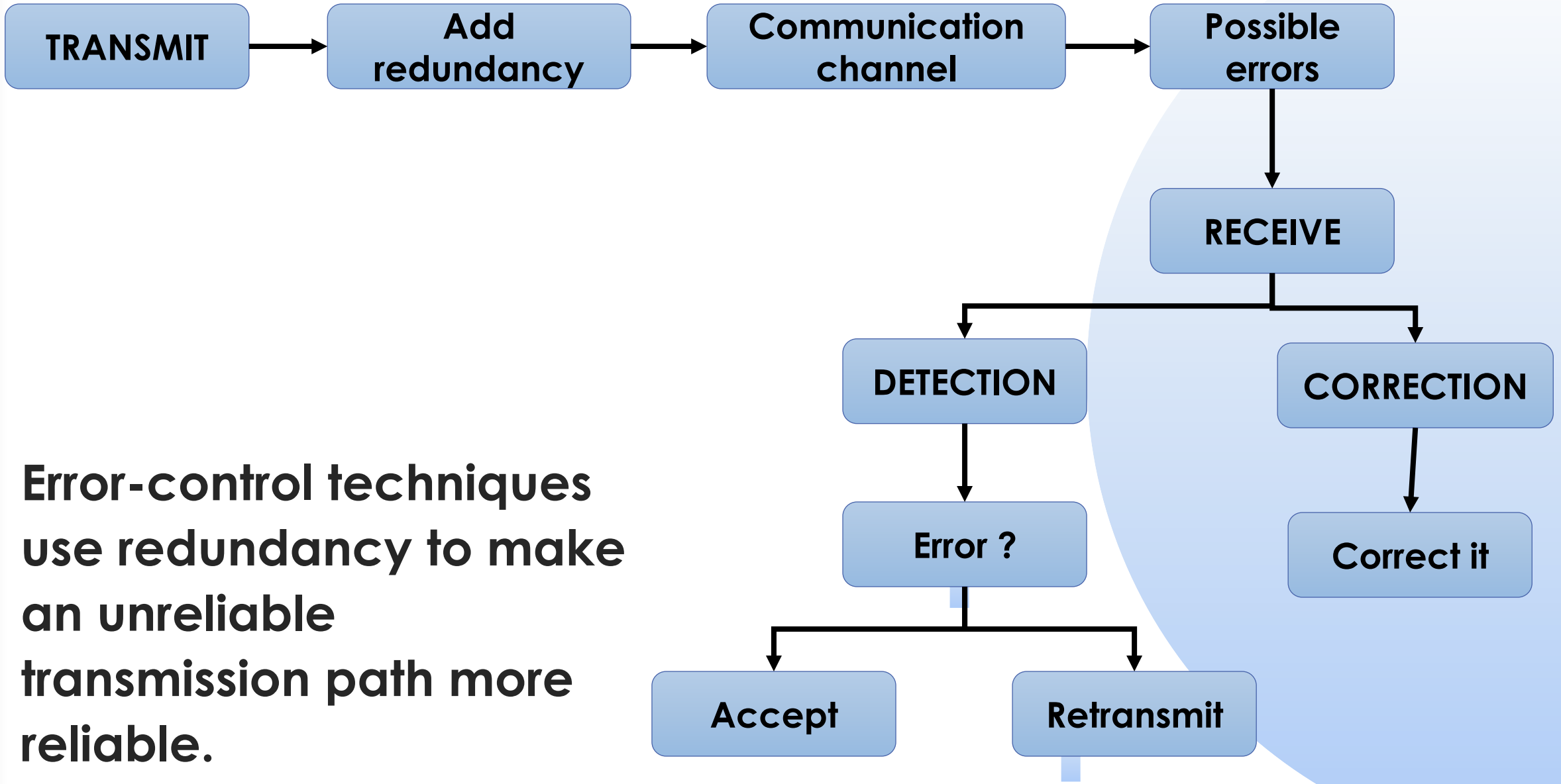
Main techniques:

- **Detection**
  - Parity
  - Checksum
  - CRC
- **Correction**
  - Two-dimensional parity
  - Block error-correcting codes
  - FEC

Which Technique Should We Use?

Situation	Suitable approach
Simple error detection	Parity
Transport-layer error detection	Checksum
Strong link-layer error detection	CRC
Receiver must correct errors immediately	FEC
Reliable channel + occasional errors	Detection + retransmission
Noisy / long-delay channel	FEC

Big Picture



Error-control techniques use redundancy to make an unreliable transmission path more reliable.

Review Questions

1. What is a bit error?
2. What are the two general types of transmission errors?
3. What is the difference between a single-bit error and a burst error?
4. Why can the same noise event affect more bits at a higher data rate?
5. Why is redundancy added to transmitted data?
6. What is the difference between error detection and error correction?
7. What is Forward Error Correction (FEC)?
8. What is the basic idea behind backward error control?
9. Why is FEC particularly useful for noisy or long-delay links?
10. Why can error detection followed by retransmission be efficient on reliable links?

Review Questions

11. What is a parity bit?
12. Explain even parity and odd parity.
13. For the data 1011010, what is the even parity bit?
14. Why can a single parity bit fail to detect a two-bit error?
15. How does two-dimensional parity improve upon single parity?
16. How can two-dimensional parity correct a single-bit error?
17. What is the basic principle of a checksum?
18. What is the Internet checksum?
19. Why does TCP/UDP use a checksum?
20. Why is checksum generally considered weaker than CRC?

Review Questions

21. What is CRC?
22. What is a generator in CRC?
23. Why are zeros appended to the data before CRC calculation?
24. What type of arithmetic is used in CRC?
25. Why is XOR used in CRC calculations?
26. What does a nonzero remainder at the receiver indicate?
27. Why is CRC more powerful than a simple parity check?
28. What is an error-correcting code?
29. What is Hamming distance?

Review Questions

30. Calculate the Hamming distance between:

    101101

    100001

31. Why does increasing the distance between valid codewords help error correction?

32. What is the basic principle of minimum-distance decoding?

Thank you

COM 3401

DATA COMMUNICATION AND NETWORKING

Lecture 07

ERROR DETECTION AND CORRECTION